End-to-End MLOps Pipeline for Heart Disease Prediction

Student Name: Kachhadiya Ravi
BITS ID: 2024ac05707 Course: ML Ops, AIMLCZG523
Repository: https://github.com/rkdgr8/ML-Ops

1. Project Overview

This project implements an end-to-end MLOps workflow for predicting heart disease risk using the UCI Heart Disease dataset. The objective is to build a reproducible machine learning solution that covers data acquisition, preprocessing, model training, experiment tracking, API serving, testing, containerization, and deployment readiness.

The model predicts whether a patient is likely to have heart disease based on clinical attributes such as age, resting blood pressure, cholesterol, maximum heart rate, exercise-induced angina, ST depression, number of major vessels, and thalassemia category.

2. Dataset

Dataset used: UCI Heart Disease dataset, Cleveland processed data.

The dataset contains 14 columns:

Column	Description
`age`	Patient age
`sex`	Patient sex
`cp`	Chest pain type
`trestbps`	Resting blood pressure
`chol`	Serum cholesterol
`fbs`	Fasting blood sugar indicator
`restecg`	Resting ECG result
`thalach`	Maximum heart rate achieved
`exang`	Exercise-induced angina
`oldpeak`	ST depression induced by exercise
`slope`	Slope of peak exercise ST segment
`ca`	Number of major vessels
`thal`	Thalassemia category
`target`	Heart disease diagnosis

The original target values were converted into a binary classification target:

Value	Meaning
0	No heart disease
1	Heart disease present

Missing values represented by `?` were converted to null values and removed from the cleaned training dataset.

3. Data Acquisition and Preprocessing

The data pipeline is implemented in `src/data_processing.py`.

The pipeline performs the following actions:

- downloads the Cleveland dataset from the UCI repository
- assigns meaningful column names
- converts `?` values into missing values
- converts `ca` and `thal` into numeric columns
- drops rows containing missing values
- converts the original multi-class target into binary classification
- saves the cleaned dataset to `data/processed/heart_processed.csv`

Command used:

```
python -m src.data_processing
```

4. Exploratory Data Analysis

The EDA workflow is implemented in `src/eda.py`.

The cleaned dataset contains 297 rows and 14 columns after removing records with missing values. No missing values remain in the processed dataset.

Target distribution:

Target	Meaning	Count
0	No heart disease	160
1	Heart disease present	137

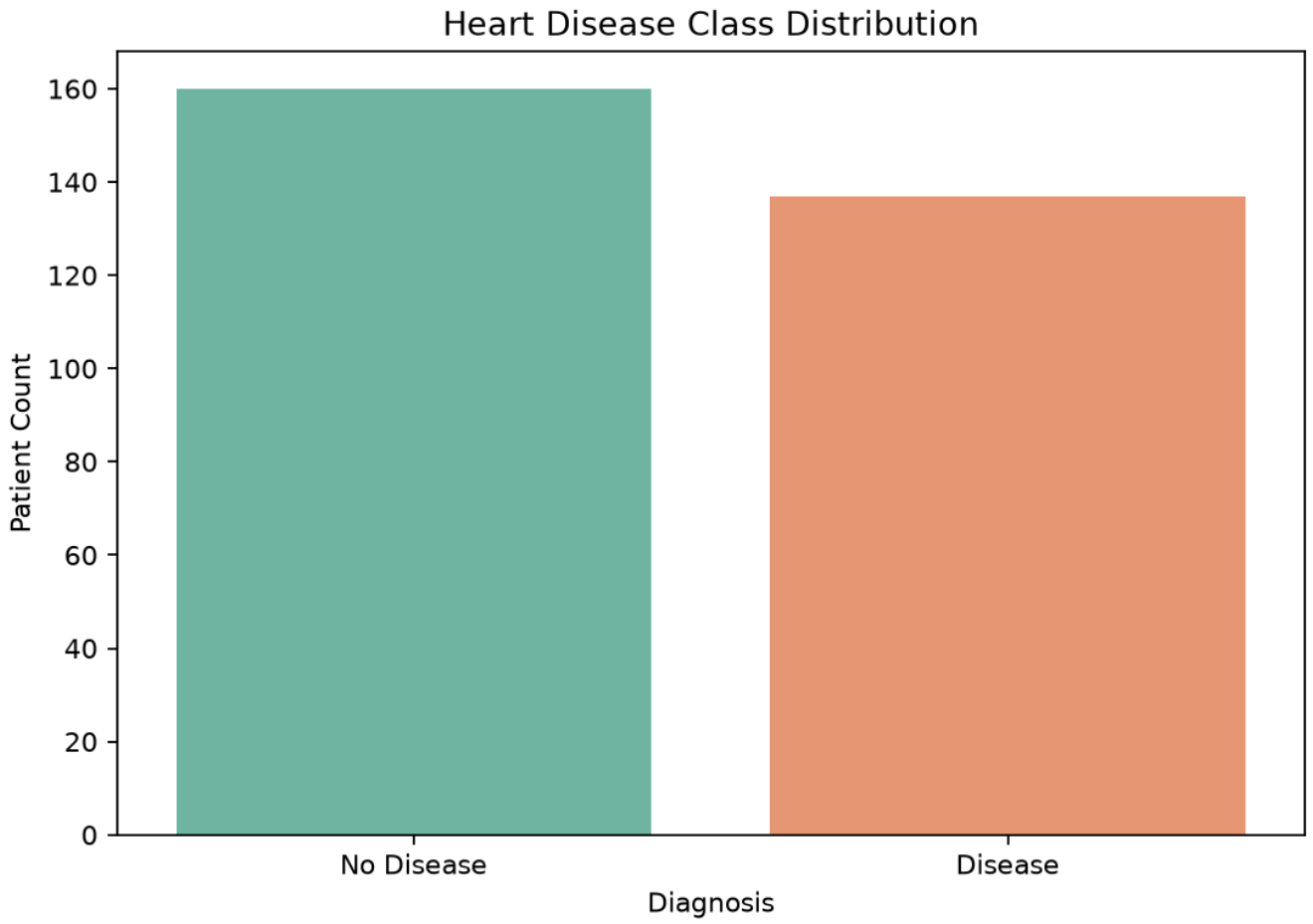
EDA outputs generated:

- class distribution plot
- numeric feature histograms
- correlation heatmap
- missing value analysis
- age versus maximum heart rate relationship

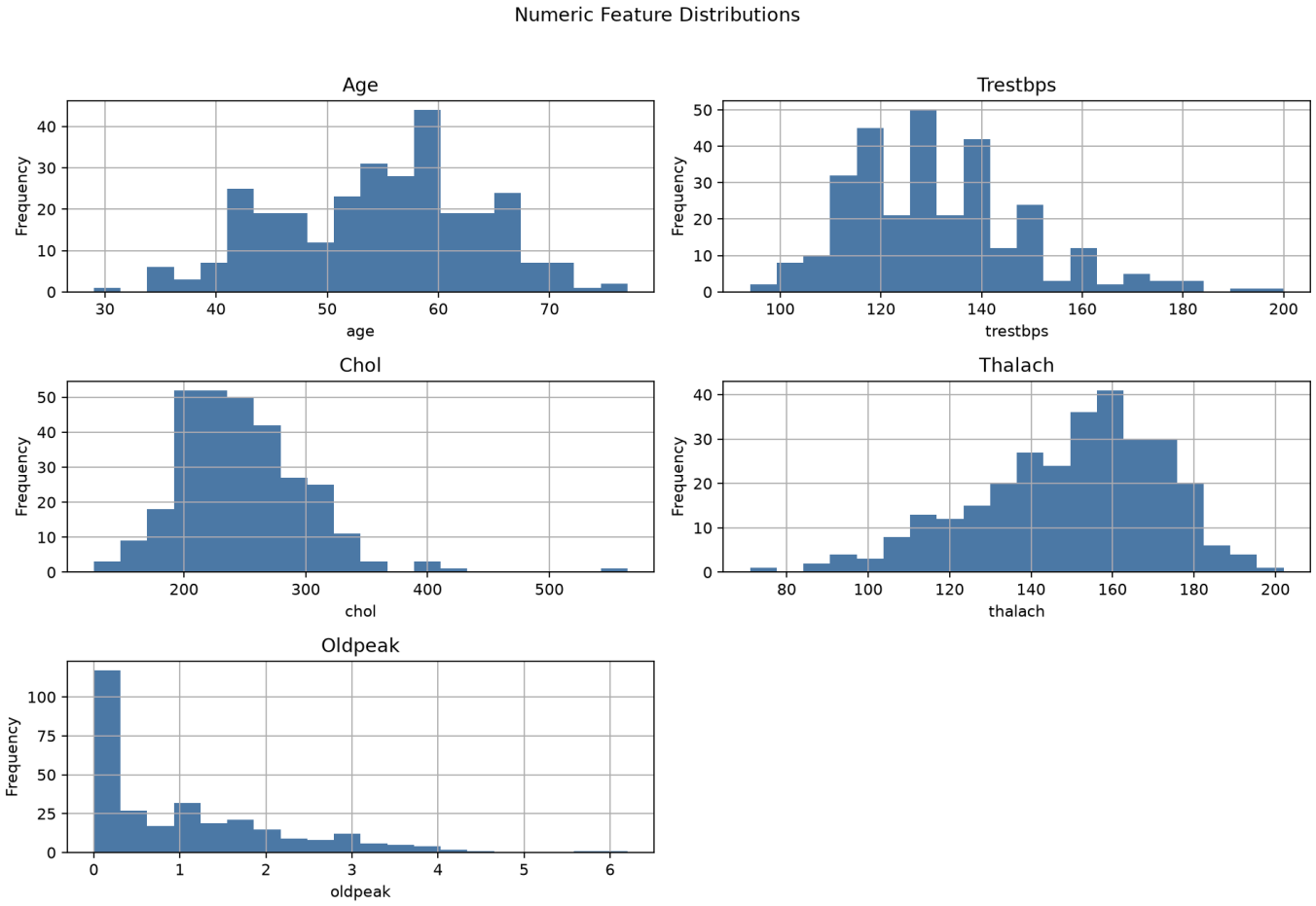
Command used:

```
python -m src.eda
```

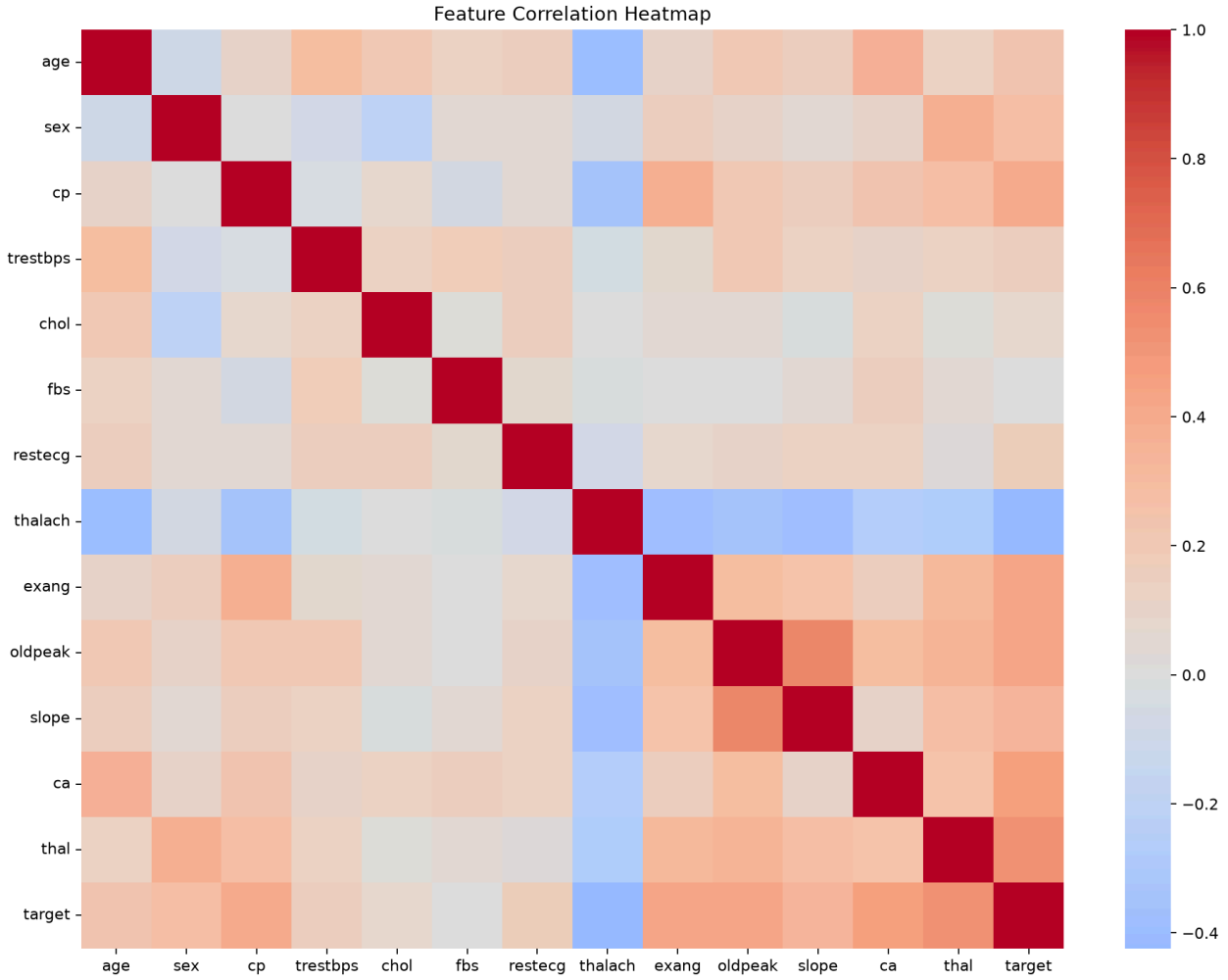
Class distribution:



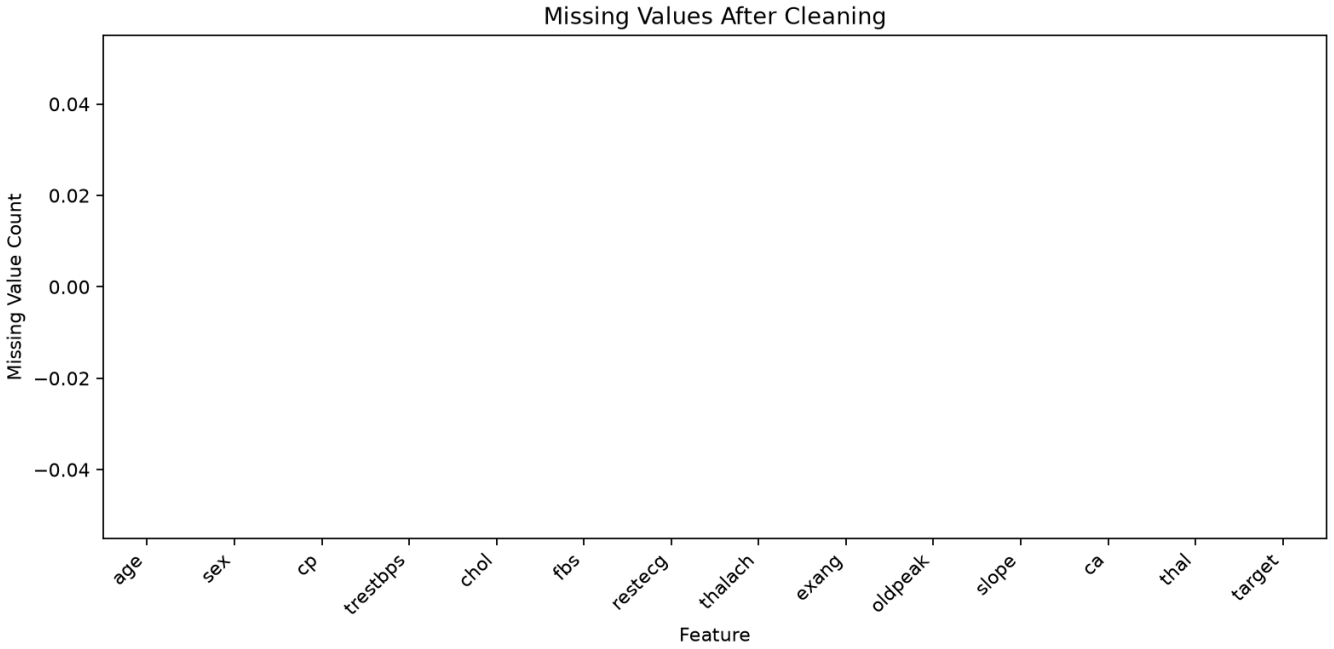
Numeric feature distributions:



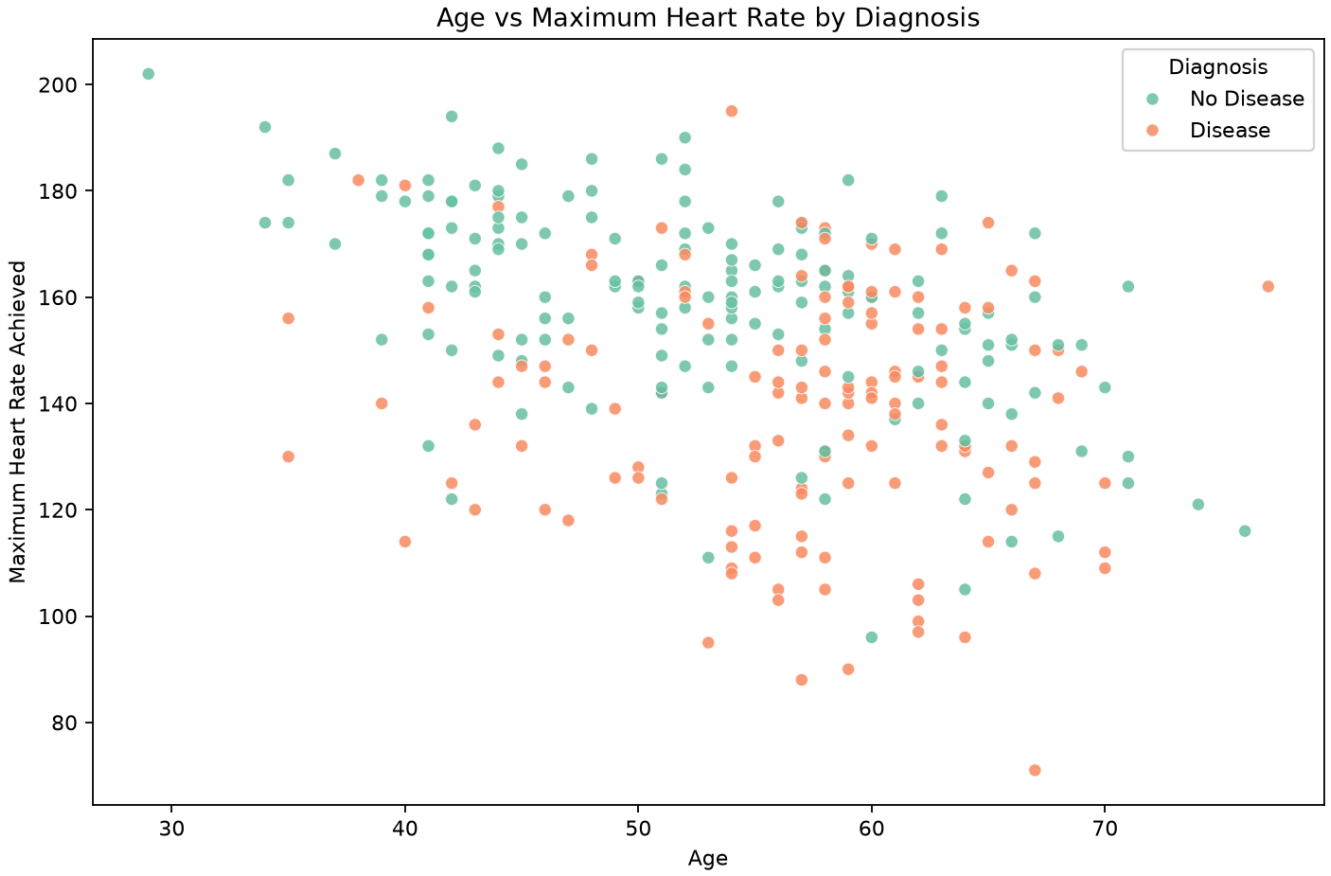
Correlation heatmap:



Missing value analysis:



Age versus maximum heart rate:



5. Model Development

The training workflow is implemented in `src/train.py`.

Two classification models were trained and compared:

- Logistic Regression
- Random Forest

The preprocessing and model were packaged together using a Scikit-learn `Pipeline`. The preprocessing stage uses:

- `StandardScaler` for numeric features
- `OneHotEncoder` for categorical features
- `ColumnTransformer` to combine both preprocessing branches

Hyperparameter tuning was performed using `GridSearchCV` with five-fold cross-validation and ROC-AUC as the optimization metric.

Command used:

```
python -m src.train
```

6. Model Evaluation

The best model selected was:

```
logistic_regression
```

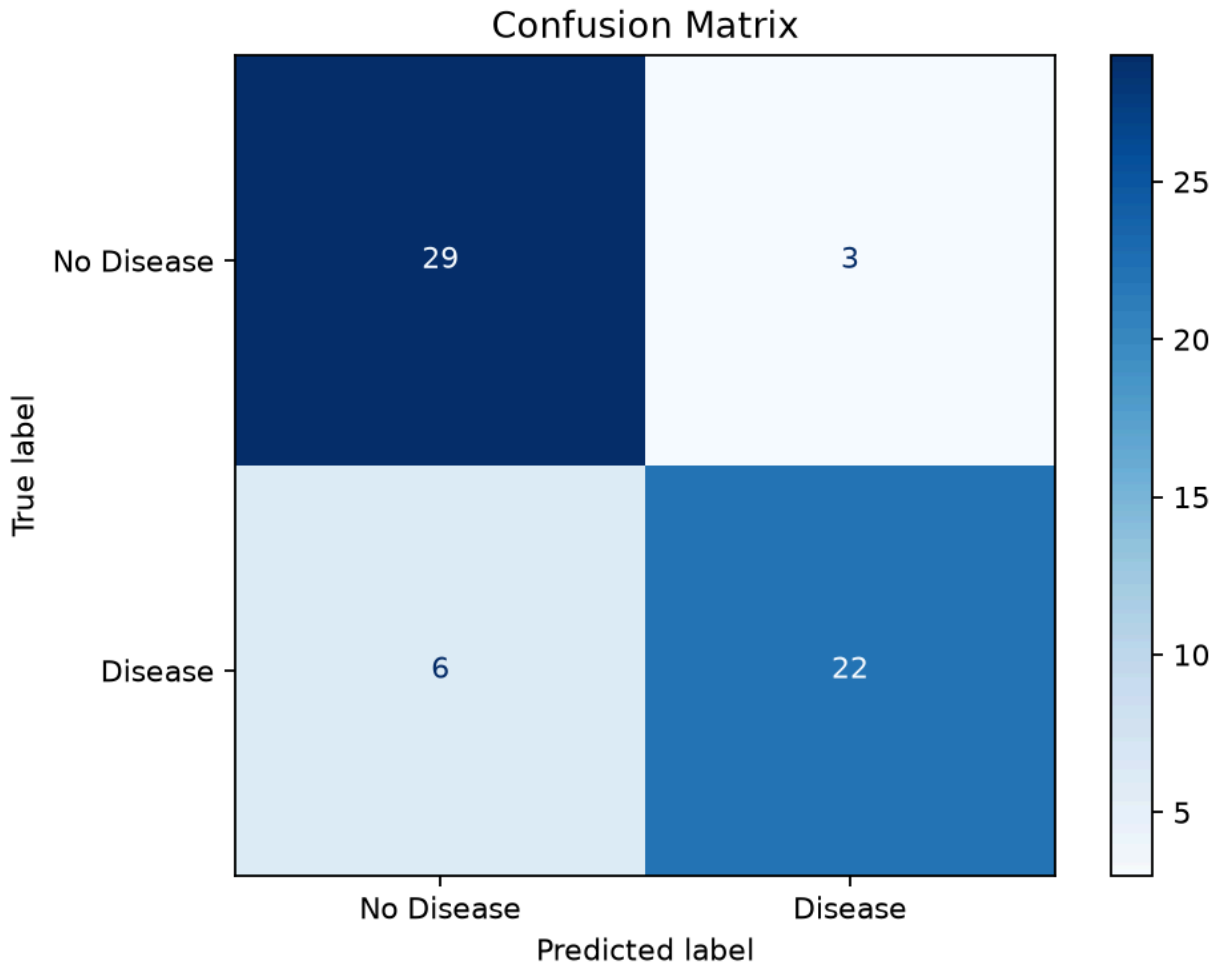
Model comparison:

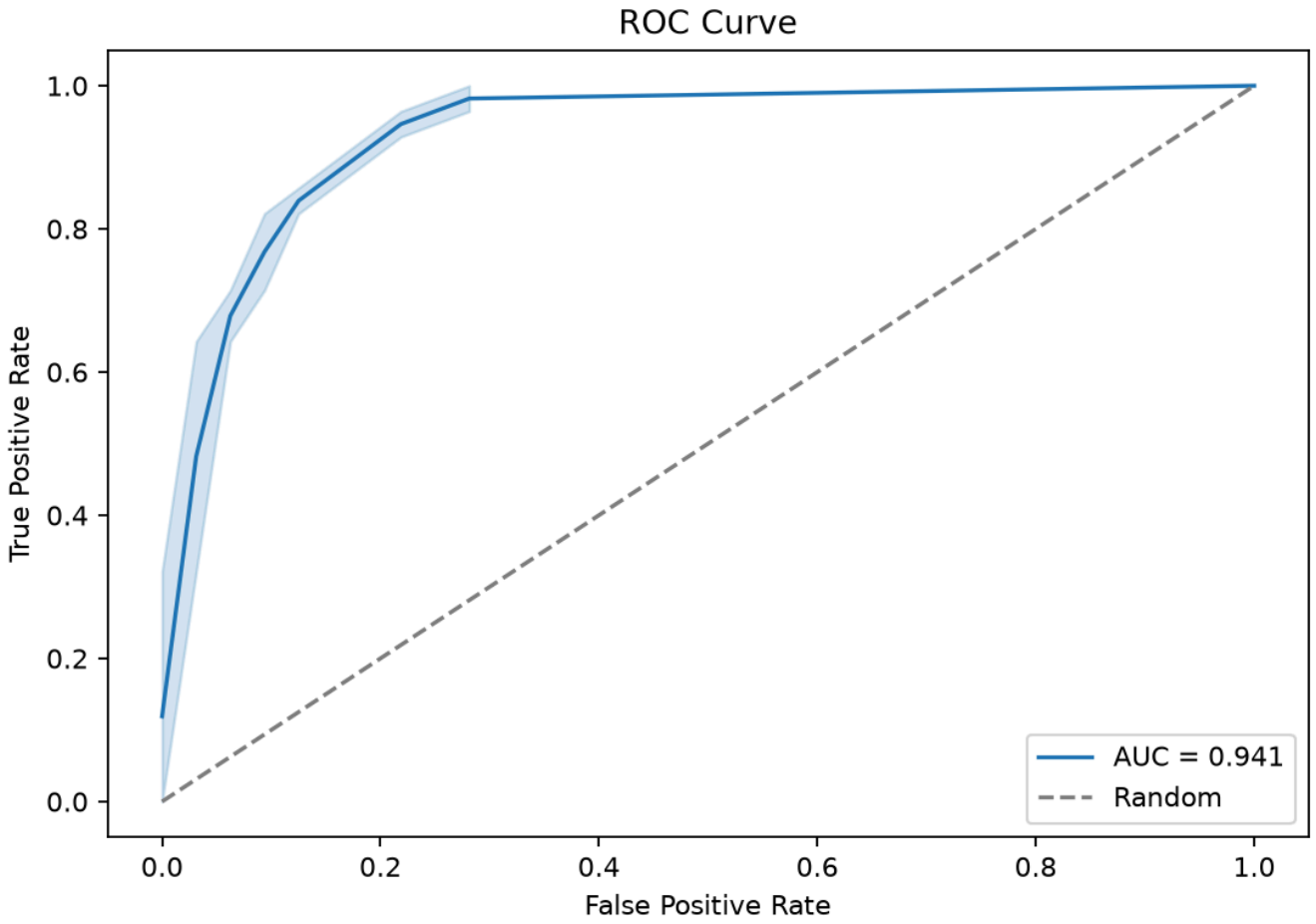
Model	CV ROC-AUC Mean	Accuracy	Precision	Recall	F1	Test ROC-AUC
Logistic Regression	0.9052	0.8500	0.8800	0.7857	0.8302	0.9408
Random Forest	0.9004	0.8500	0.8519	0.8214	0.8364	0.9319

The final saved model artifact is:

```
models/heart_disease_pipeline.joblib
```

Model evaluation artifacts:





7. Experiment Tracking with MLflow

MLflow was integrated into the training workflow to track experiments, parameters, metrics, and model artifacts.

Experiment name:

```
heart-disease-classification
```

For each model run, the following are logged:

- model name
- best hyperparameters
- cross-validation ROC-AUC mean and standard deviation
- test accuracy
- test precision
- test recall
- test F1-score
- test ROC-AUC
- trained model artifact
- confusion matrix and ROC curve for the best model
- metrics JSON artifact

Command to start MLflow UI:

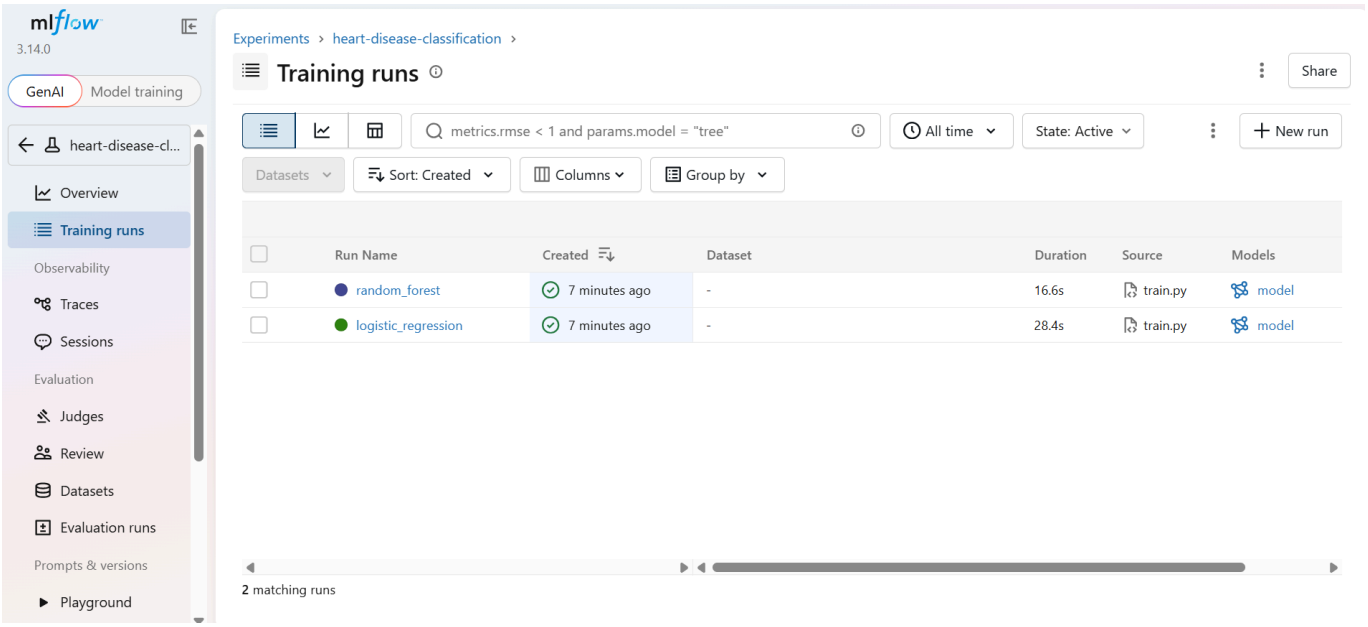
```
mlflow ui
```

MLflow UI URL:

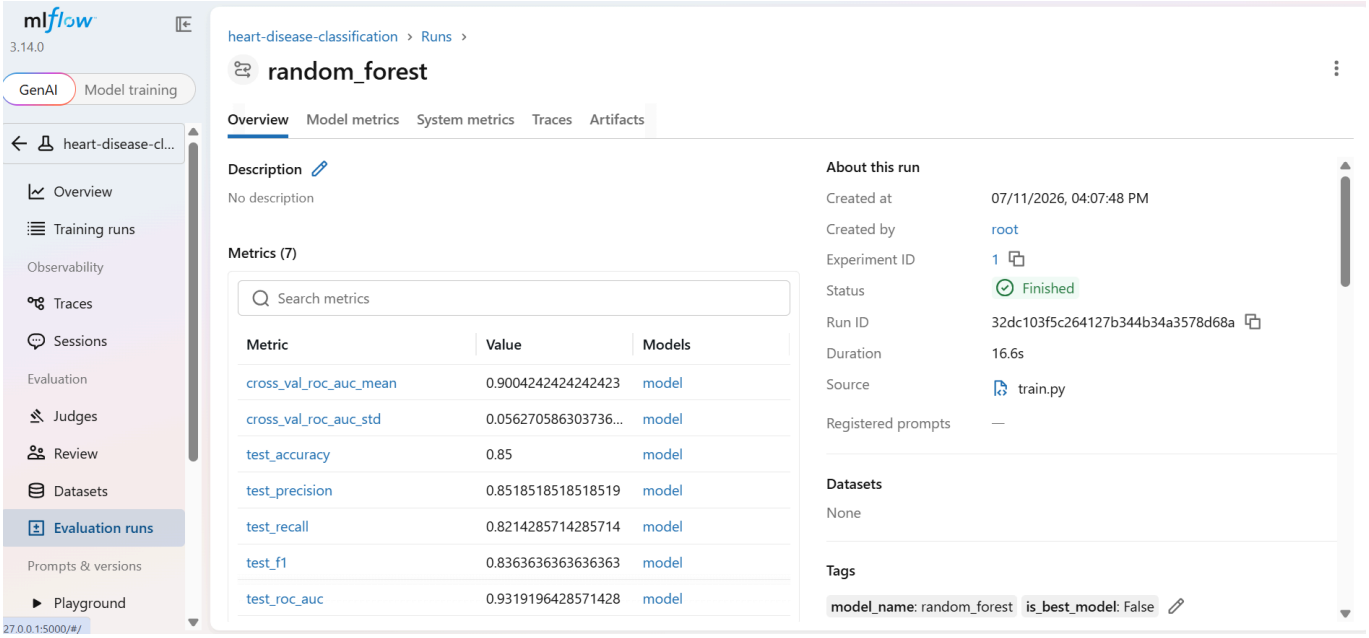
```
http://127.0.0.1:5000
```

The MLflow UI can be used to inspect and compare the logged experiment runs.

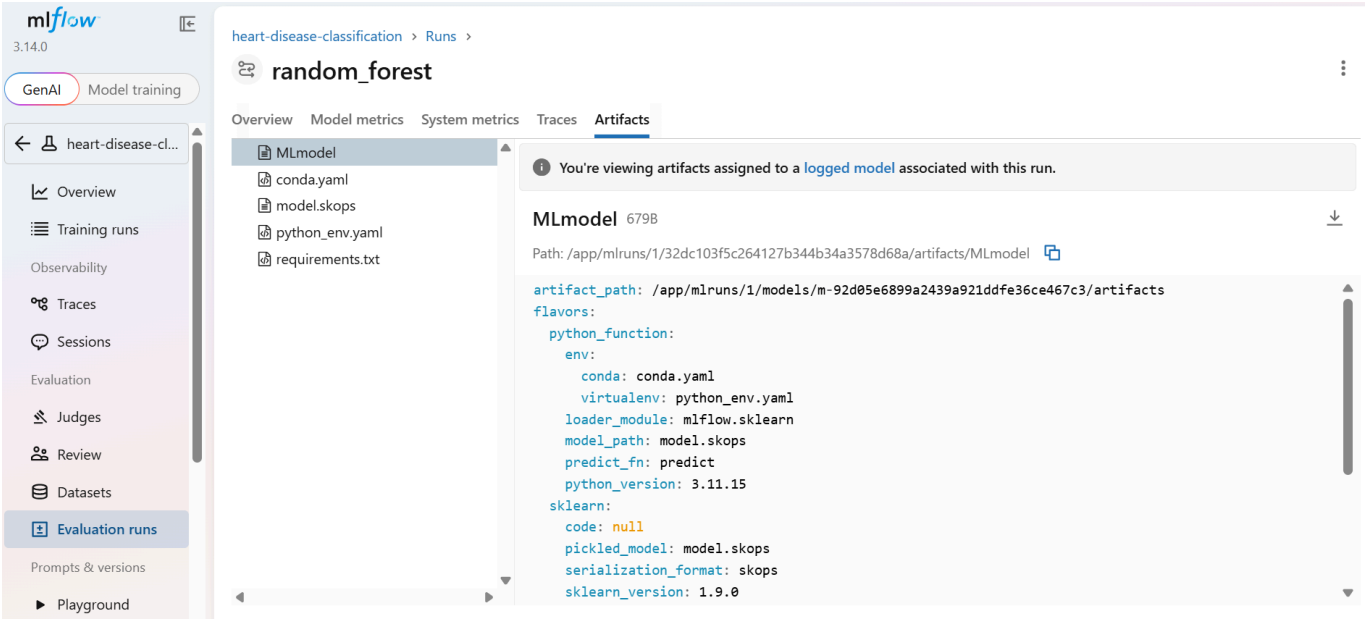
MLflow training runs:



Random Forest run details:



Logged model artifacts:



8. API Serving

The prediction API is implemented using FastAPI in `api/main.py`.

Available endpoints:

Endpoint	Method	Purpose
`/health`	GET	Health check
`/predict`	POST	Predict heart disease risk
`/docs`	GET	Swagger UI

The `/predict` endpoint accepts patient attributes as JSON and returns:

- binary prediction
- confidence score
- diagnosis label

Example request:

```
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d "
{\"age\":67,\"sex\":1,\"cp\":4,\"trestbps\":160,\"chol\":286,\"fbs\":0,\"restecg\"
:2,\"thalach\":108,\"exang\":1,\"oldpeak\":1.5,\"slope\":2,\"ca\":3,\"thal\":3}"
```

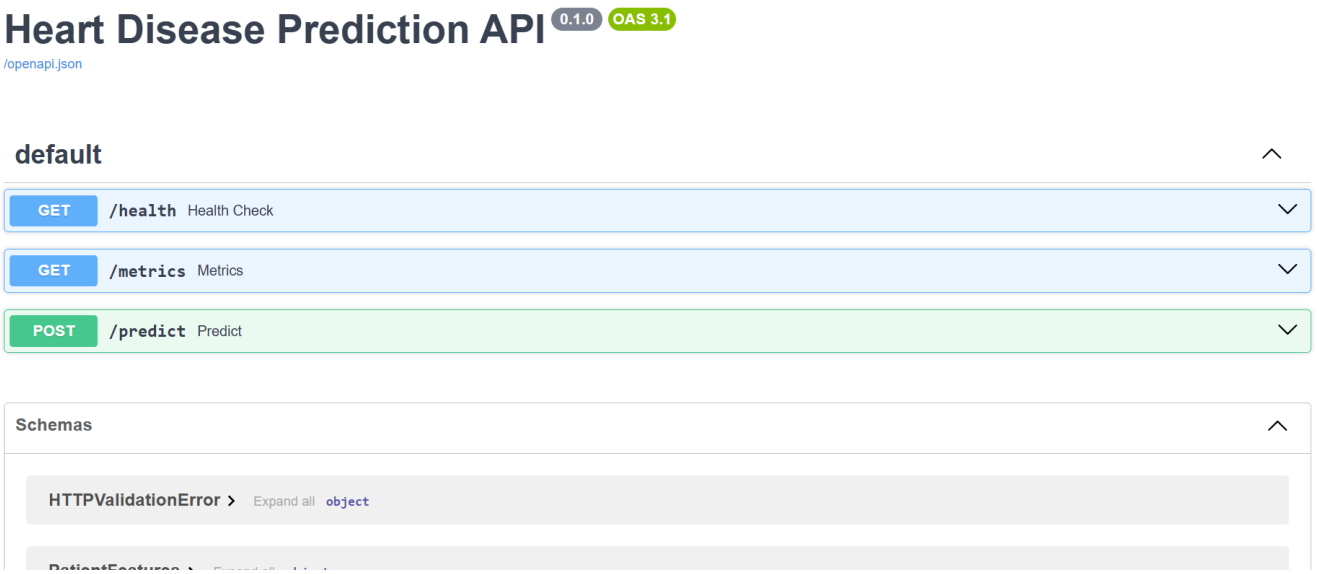
Example response:

```
{
  "prediction": 1,
  "confidence": 0.8964416521576047,
```

```
  "diagnosis": "heart disease risk"
}
```

The API was tested locally through Docker using `/health`, `/predict`, and `/metrics`.

Swagger UI:



Prediction example in Swagger:

9. Testing

Unit tests were added for:

- data cleaning
- EDA summary generation
- preprocessing pipeline construction
- training helper functions

- MLflow logging orchestration
- API health check
- API prediction response contract
- Prometheus metrics endpoint

Test command:

```
pytest
```

Docker-based test command:

```
docker run --rm -v "D:\BITS\courses\sem3\MLOps\Assigment:/app" -w /app heart-disease-api pytest
```

Latest Docker-based local test result:

```
9 passed, 1 warning
```

10. Containerization

The API is containerized using Docker.

Build command:

```
docker build -t heart-disease-api .
```

Run command:

```
docker run --rm -p 8000:8000 heart-disease-api
```

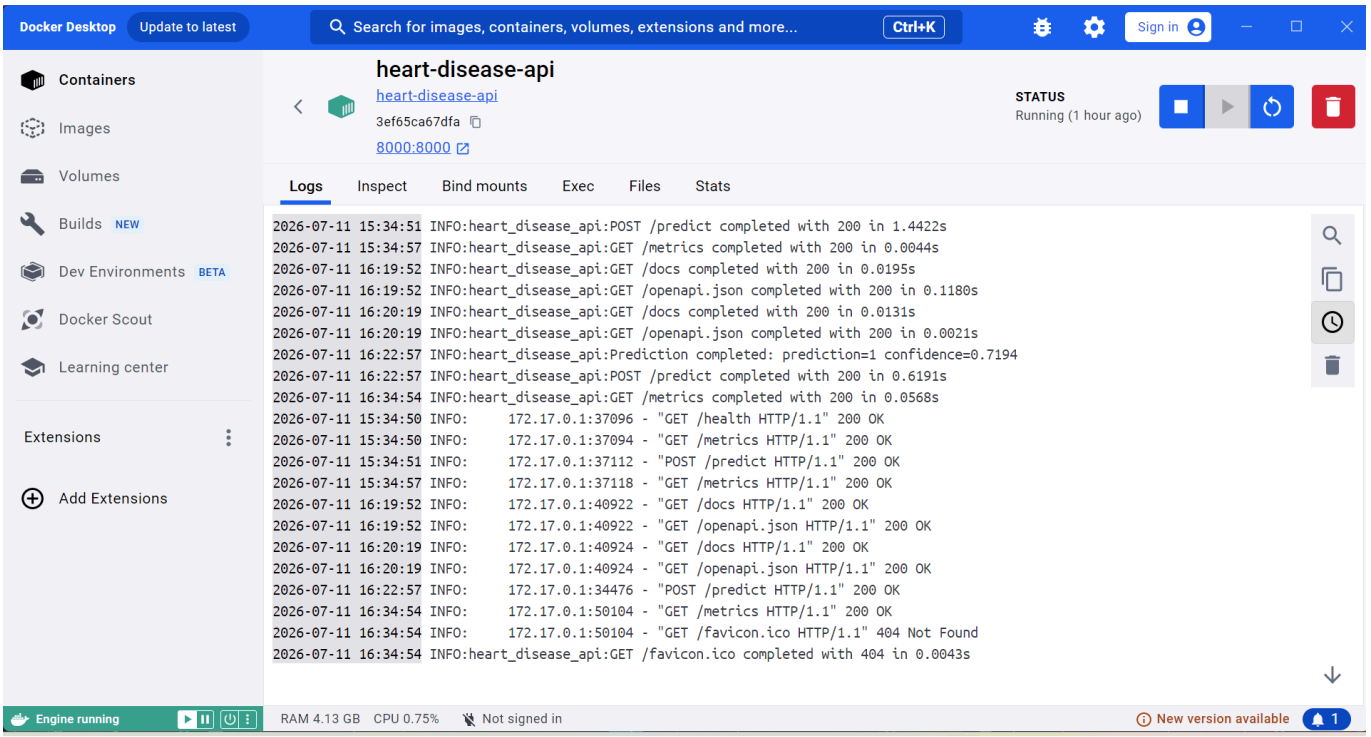
The container exposes the FastAPI service on port `8000`.

Local container validation was completed using:

```
GET /health
POST /predict
GET /metrics
```

The Docker container was run locally and exposed the service on port `8000`.

Docker running-container screenshot placeholder:



11. CI/CD

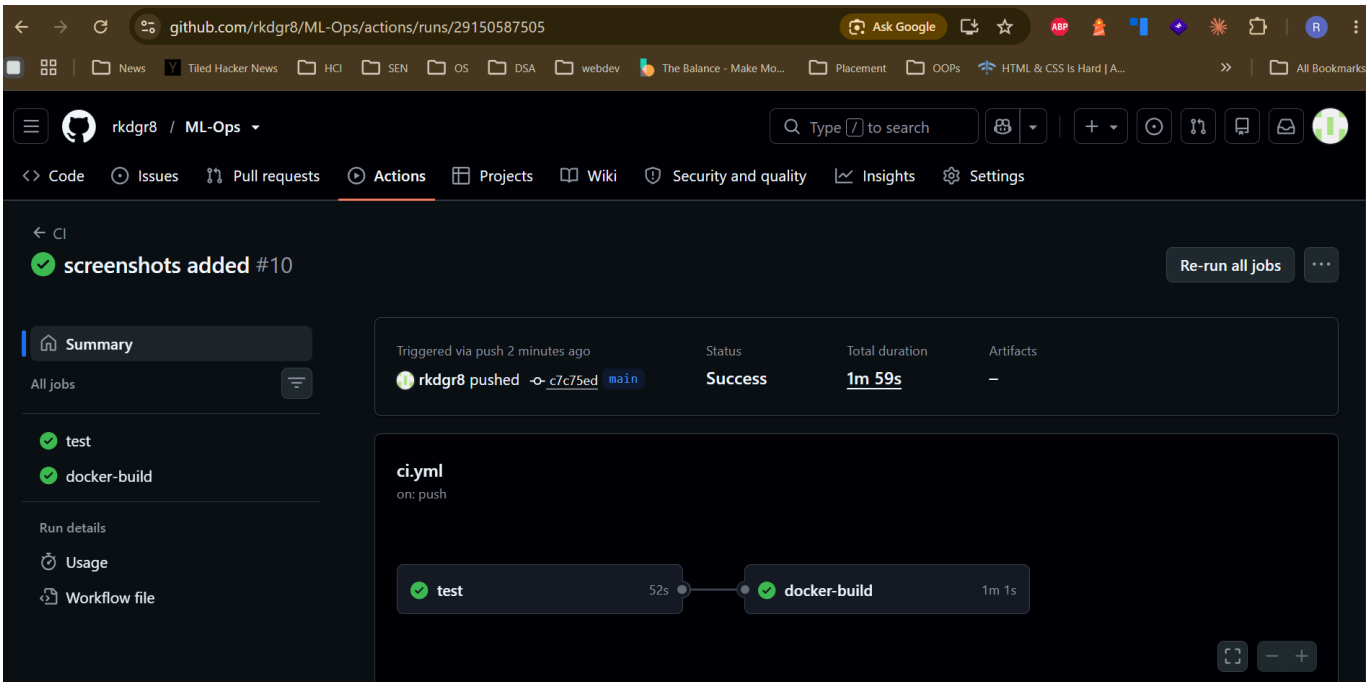
A GitHub Actions workflow is defined in `.github/workflows/ci.yml`.

The workflow performs:

- repository checkout
- Python setup
- dependency installation
- linting with Ruff
- unit testing with Pytest
- Docker image build validation

The CI workflow is triggered on push and pull request events.

GitHub Actions CI result:



12. Deployment

Kubernetes manifests are provided in the `k8s/` directory:

```
k8s/deployment.yaml
k8s/service.yaml
k8s/README.md
```

The deployment manifest runs the API container, and the service manifest exposes it through a load balancer style service.

The deployment includes:

- one API replica
- container port `8000`
- readiness probe on `/health`
- liveness probe on `/health`
- CPU and memory requests/limits
- service routing from port `80` to container port `8000`

Local deployment commands:

```
docker build -t heart-disease-api:latest .
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl get pods
kubectl get services
```

If the service is not directly reachable, port-forward it:

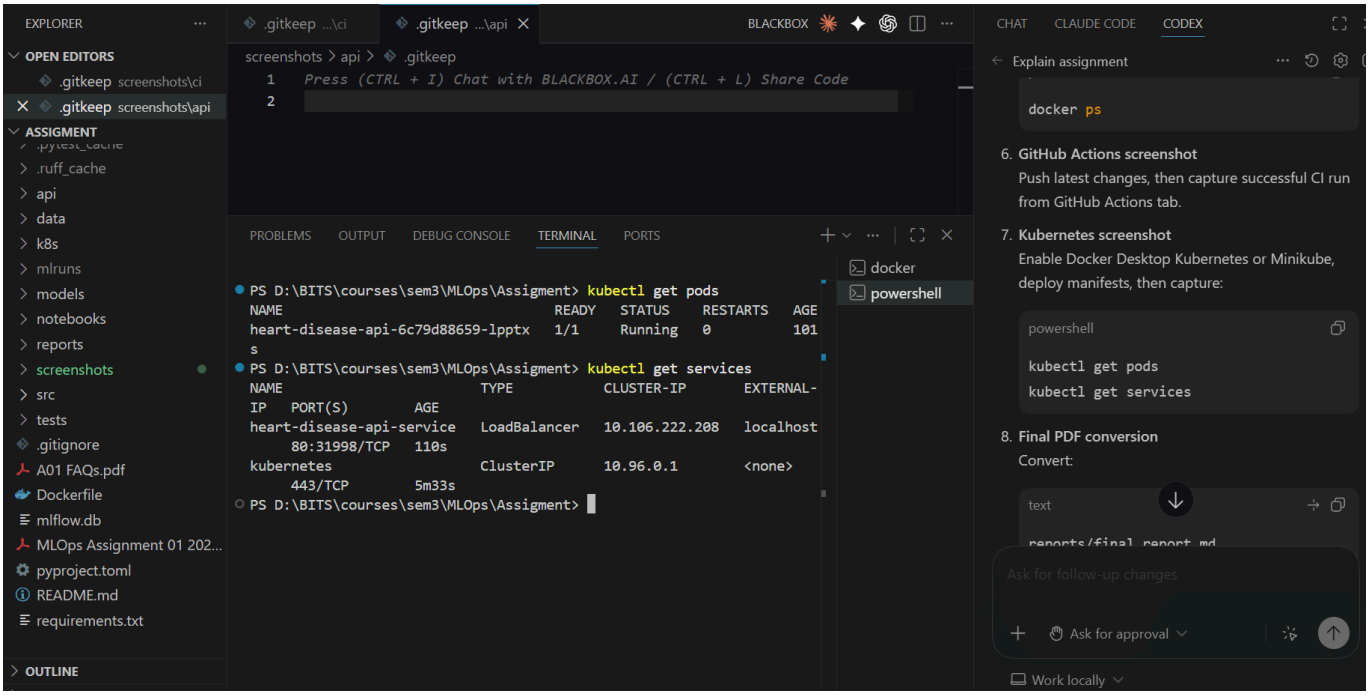
```
kubectl port-forward service/heart-disease-api-service 8000:80
```

Kubernetes deployment could not be executed locally at this stage because no active Kubernetes context was configured on the machine:

```
kubectl config current-context
error: current-context is not set
```

Docker Desktop Kubernetes or Minikube should be enabled before final submission to capture pod and service evidence screenshots.

Kubernetes deployment screenshot:



13. Monitoring and Logging

Current monitoring support includes:

- API health check endpoint
- prediction request validation through FastAPI/Pydantic
- container logs through Docker
- request logging middleware
- Prometheus-compatible `/metrics` endpoint
- request count, request latency, and prediction count metrics

Metrics endpoint:

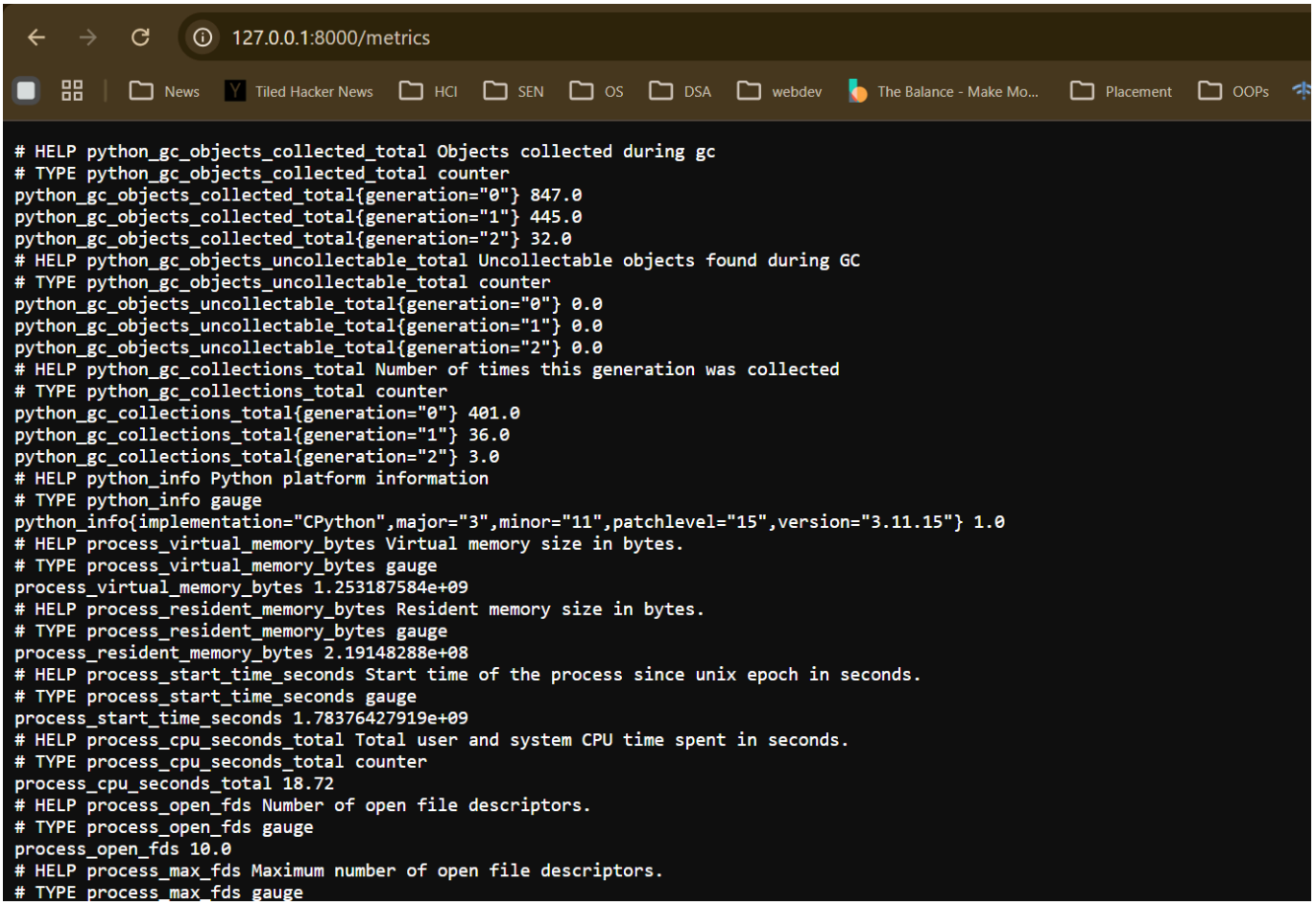
```
http://127.0.0.1:8000/metrics
```

The `/metrics` endpoint provides Prometheus-compatible metrics. Optional future enhancement: connect Prometheus and Grafana for dashboard screenshots.

Example metrics exposed:

```
api_requests_total
api_request_latency_seconds
model_predictions_total
```

Metrics endpoint:



14. Conclusion

This project demonstrates a practical MLOps workflow for a binary classification problem. The solution includes reproducible data preparation, model training, experiment tracking, model packaging, API serving, Docker containerization, tests, and deployment manifests.

The Logistic Regression model achieved the best ROC-AUC score on the test set and was selected as the final production model.

15. Evidence To Attach Before Final Submission

The implementation is complete for the major MLOps components. Before final PDF submission, attach the following screenshots in the relevant sections:

Evidence	Status
EDA plots	Generated in `screenshots/eda/`
Model confusion matrix and ROC curve	Generated in `screenshots/model/`
MLflow experiment UI	Added in `screenshots/mlflow/`
Swagger UI `/docs`	Added in `screenshots/swagger/swagger_docs.png`
`/predict` response	Added in `screenshots/swagger/swagger_example_request.png`
`/metrics` endpoint	Added in `screenshots/metrics/metrics.png`
Docker running container	Added in `screenshots/docker/docker_container.png`
GitHub Actions successful CI run	Added in `screenshots/ci/ci.png`
Kubernetes pods/services	Added in `screenshots/k8s/k8s.png`

16. Appendix

Repository:

```
https://github.com/rkdgr8/ML-Ops
```

Demo video:

```
https://drive.google.com/file/d/10Tf_SN1QJ6JIZ3V4gYB6UTBJjmSwdgVF/view?usp=sharing
```

Important commands:

```
python -m src.data_processing
python -m src.eda
python -m src.train
uvicorn api.main:app --host 127.0.0.1 --port 8000
docker build -t heart-disease-api .
docker run --rm -p 8000:8000 heart-disease-api
pytest
```